TEK BAŞINA BAŞLA · 1/15 · mac · codex

Bir dosya. Baştan sona bir rota.

Sunum öğretir; bu rehber bilgisayarında işi yaptırır.

01	02	03	04
Çalışan ortam	**Kaynaklı bağlam**	**İncelenmiş plan**	**Kanıtlı kapanış**
Terminal → CLI → doğru proje	Mobil + API + backend + karar	Proposal → spec → design → tasks	Uygula → sına → sync → archive

Önce rehber panelinden işletim sistemini ve asistanını seç. Soldaki adımları sırayla tamamla. Her adımın sonunda **“Geçiş kanıtı”** var: yalnız okumuş olmak yeterli değil.

Bu eğitimle çıkaracağın iş

Kendi ürününde tek akış için kaynak sicili, açık kararlar, bir değişiklik planı ve kabul kanıtı. Kod yazmak PM’in ön koşulu değil; uygulama ve test aşamasını mühendislikle yürütürsün.

Yanında bulunacak malzeme

Bilgisayar ve terminal, Node/npm, erişimi olan bir AI kod asistanı, ürünün kaynakları ve karar sahipleri. Kurulum için internet; bu rehberi okumak için internet gerekmez.

OpenSpec ne yapar? Asistana ortak bir çalışma yöntemi ve dosya düzeni verir. Beklenen davranış ile önerilen değişikliği sürümlenen metinlerde tutar. AI hesabı, ürün bilgisi, kod erişimi, test ortamı ve release otomatik oluşmaz.

TERMINAL = komut çalıştır	ASİSTAN SOHBETİ = mesaj gönder	DOSYA = oku / düzenle	EKİP = kararı incele

Bu rehber PM’in ilk işi ve günlük kullanımı içindir. İleri seçenekler atlas içinde. Kaynak linkleri doğrulama içindir; gerekli açıklamalar bu dosyada bulunur.

Geçiş kanıtı
Hangi projede, hangi asistanla, hangi tek kullanıcı sonucunu ele alacağını bir cümleyle yaz.

10 Ekim 2026 · Terminal ve sohbet ayrımı ↗

TEK BAŞINA BAŞLA · 2/15 · mac · codex

Önce komutları çalıştıran ortam.

Node, npm ve terminal: üç adı ezberlemek yerine işlerini bil.

Terminali aç

macOS: ⌘ Space → Terminal → Enter.
Windows: Başlat → PowerShell veya Windows Terminal.
Linux: uygulama menüsü → Terminal.

Bir komutu yapıştırıp Enter'a bas. Sonuç gelmeden sonraki satıra geçme. Başındaki `$` terminal işaretiyse yazılmaz; `$openspec–…` ise Codex sohbetindeki skill adının parçasıdır.

Node ve npm

Node, OpenSpec'in çalıştığı JavaScript ortamı. **npm**, OpenSpec paketini indirip kuran araç. OpenSpec için Node **20.19.0 veya daha yeni** gerekir; desteklenen LTS kullan.

TERMINAL · her klasörde çalışabilir

```
node --version
npm --version
```

Beklenen sonuç

Node için `v…`, npm için bir sürüm numarası. Örneğin Node `v22.x` asgari koşulu geçer; `v20.18` geçmez.

▼ **Node yoksa nasıl kurarım?**

1. Resmi Node indirme sayfasını aç; LTS sürümünü ve işletim sistemini seç.
2. macOS için Installer/.pkg, Windows için Installer/.msi indir. Cihaz mimarini seç: Mac → Bu Mac Hakkında, Windows → Ayarlar → Sistem → Hakkında. Kurucuyu aç, adımları tamamla; npm bileşenini dahil et.
3. Terminali kapatıp yeni terminal aç. İki sürüm komutunu tekrar çalıştır.
4. Linux veya ekipçe yönetilen bilgisayarda mevcut Node yöneticisini/kurulum yöntemini koru. Sürüm eskiyse o yöntemle desteklenen LTS'ye geç; kurum kurulum yetkisi gerekiyorsa BT'den Node+npm iste.

nvm/fnm/asdf/Volta zaten varsa ikinci bir yöntem kurma. Aktif Node sürümü değişince global CLI'ın görünürlüğünü yeniden kontrol et.

Asistan da hazır olmalı

Halihazırda kullandığın Codex / Claude Code / Cursor hesabında oturum aç. Uygulamada bir yerel proje açıp basit bir dosyayı okuyabildiğini doğrula. Sıradan web sohbetine repo URL'si yapıştırmak yerel dosya erişimi sağlamaz.

▼ AI kod asistanım henüz yok

Bu eğitimin varsayılanı Codex. Resmi Codex girişinden masaüstü uygulamasını indir, kur, oturum aç ve yerel klasörünü proje olarak aç. Kurum hesabında izin verilen asistanı kullan. Claude Code veya Cursor zaten varsa üstte onu seç; komutlar ona uyarlanır.

CLI tercih eden macOS/Linux kullanıcıları için resmi Codex kurucusu ve ilk açılış:

TERMINAL · isteğe bağlı Codex CLI

```
curl -fsSL https://chatgpt.com/codex/install.sh | sh
codex
```

İlk açılışta sunulan giriş seçeneğiyle hesabına gir. OpenSpec kurulumu bu asistan kurulumundan ayrıdır.

Geçiş kanıtı
Node/npm sürümleri koşulu geçiyor ve seçilen asistan hesabı bir yerel dosyayı okuyabiliyor.

10 Ekim 2026 · Runtime ve kurulum ↗ · Codex kurulum kaynağı ↗

TEK BAŞINA BAŞLA · 3/15 · mac · codex

Kullanıcı hesabına CLI kur.

Global, “bütün ürünlerin bağlamı ortak” demek değil; komutun her klasörden çağrılabilmesi demek.

Aşağıdaki komutlar seçtiğin işletim sistemine göre değişir. Mevcut Node yöneticin çalışıyorsa **standart kurulumu** seç. İzin hatasında home alternatifi kendi kullanıcı klasörüne kurar; sistem dizinine yazmaz.

Standart kurulum

TERMINAL · her klasörde · satırları sırayla çalıştır

```
npm install -g @fission-ai/openspec@latest
openspec --version
openspec --help
```

Home alternatifi

TERMINAL · standart global hedefte izin sorunu varsa

```
npm install -g @fission-ai/openspec@latest --prefix "$HOME/.local"
export PATH="$HOME/.local/bin:$PATH"
openspec --version
```

Kalıcı PATH: `echo "$SHELL"` ile shell’i gör. zsh ise `~/.zshrc`, bash ise `~/.bashrc` dosyasını metin editöründe aç/oluştur; `export PATH="$HOME/.local/bin:$PATH"` satırını bir kez ekle. macOS bash login oturumunda `~/.bash_profile` bu dosyayı yüklemiyorsa aynı satırı onun içine koy. Yeni terminal aç; `command -v openspec` ve sürümü kontrol et. GUI asistanı başka PATH kullanıyorsa uygulamayı yeniden açıp orada da kontrol et.

Kurulum dizini ≠ kişisel ayar

TERMINAL · yer gösterir, kurulumu başlatmaz

PATH ne?

Terminalin çalıştırılabilir program aradığı dizin listesi. Home kurulumunda komut dosyası bu listeye eklenir. `export` yalnız mevcut terminal içindir;

```
npm prefix -g
openspec config path
```

`npm prefix -g` npm'in varsayılan global hedefini gösterir. Tek komutta kullanılan `--prefix` bu varsayılanı değiştirmez. `openspec config path` kullanıcıya ait ayar dosyasının konumunu gösterir.

kalıcı eklemeyi sağdaki/aşağıdaki açıklamaya göre yap.

Ürün bağlamı daha sonra `proje/openspec/config.yaml` içinde tutulur. CLI'ı home'a kurmak bütün ürünlerin planını home'da tutmayı gerektirmez.

Geçiş kanıtı
Yeni açılmış terminalde openspec --version ve openspec --help çalışıyor.

10 Ekim 2026 · Kurulum ↗ · Config ve CLI ↗

TEK BAŞINA BAŞLA · 4/15 · mac · codex

Dosyaların yazılacağı kökü seç.

Root = bu işin ana klasörü. Mobil ve backend ayrıysa bunu açıkça seç.

A	B	C
Tek repo	**Ayrı repolar**	**Kod erişimi yok**
Mobil ve backend aynı repo'da: mevcut repo kökü.	PM için planlama repo'su; mobil/backend ayrıca erişilebilir.	Planlama klasörü + API/ekran/kural/test çıktısı. Görülmeyen kod "bilinmiyor".

Mevcut ekip düzeni varsa onu kullan. Yeni PM pilotunda ayrı planlama klasörü basit bir başlangıçtır. Klasör adları örnektir; gerçek yollarını yaz. Ürün A ve ürün B aynı root'ta karışmasın.

TERMINAL · yeni planlama klasörü örneği; mevcut ekip root'u varsa ona geç

```
mkdir -p "$HOME/work/product-plans"
cd "$HOME/work/product-plans"
pwd
```

`pwd` veya `Get-Location` çıktısı, komutun hangi klasörde çalıştığını gösterir. Bu adımdan sonraki `openspec` komutları bu kökte çalışır. Home (~) ürün kökü değildir.

▼ **Git / repo / clone ilk kez duyuyorum**

Repo, dosyaların sürüm geçmişiyle tutulduğu proje. **Clone**, uzaktaki repo'nun bilgisayarında çalışma kopyasını oluşturur. **Branch**, aynı repo içinde ayrı çalışma hattı. **Commit**, belirli bir kayıtlı sürümü tanımlar.

1. `git --version` çalıştır. Mac'te eksikse komutun önerdiği geliştirici araçlarını kur. Windows'ta resmi Git kurucusunu kullan. Yönetilen cihazda BT yöntemini izle.
2. Ekipten repo URL'sini, erişim yetkisini ve incelenecek branch'i al. URL tek başına izin vermez.

3. Asistandan aşağıdaki mesajla klonlama ve kaynak sürüm kaydı iste. Erişim reddedilirse repo sahibinden yetki al; erişemediğini “incelendi” sayma.

ASİSTAN SOHBETİ · yolları ve URL’leri doldur

```
Planlama root: <mutlak yol>.
Mobil repo URL: <URL>; checkout hedefi: <mutlak yol>; branch: <branch>.
Backend repo URL: <URL>; checkout hedefi: <mutlak yol>; branch: <branch>.
Mevcut checkout varsa onu kullan; değişikliklerini koru, reset yapma.
Yoksa erişimi doğrulayıp belirtilen yere clone et.
Her repo için branch, HEAD commit ve çalışma kopyasındaki değişiklikleri raporla.
Uygulama kodunu değiştirme. Başaramadığın repo’yu açıkça belirt.
```

Asistana klasörleri gerçekten ver

Yerel proje ayarlarında planlama klasörünü ana/primary klasör yap. Mobil ve backend’i ek klasör olarak ekle veya kullandığın IDE’nin workspace’inde aç. Bu klasörler erişim kapsamına girer; mesajda keşif sırasında kod değişikliği istemediğini belirt. Her kaynağın okunabildiğini bir sonraki adımda kanıtlat.

CLI kullanıyorsan önce planlama köküne geçip asistanını başlat. Başka bir root’ta açık sohbet, terminalde `cd` yaptın diye otomatik taşınmış sayılmaz.

Geçiş kanıtı
Planlama, mobil ve backend yolları; branch/commit; erişilemeyen kaynaklar tek kayıtta görünüyor.

TEK BAŞINA BAŞLA · 5/15 · mac · codex

Bu projeye OpenSpec'i bağla.

CLI kurulu. Şimdi asistanın bu root'ta OpenSpec yöntemini bulmasını sağla.

TERMINAL · seçtiğin planlama root'unda

```
openspec init ––tools codex
openspec list
```

Terminaldeki `init`, projeye OpenSpec düzenini ve seçtiğin asistanın yönerge dosyalarını ekler. Uygulama kodunu üretmez. Özel bir OpenSpec sohbet ekranı açmaz.

Dosya kontrolü

Proje ağacında `openspec/` ve asistan için üretilen skill/komut dosyalarını gör. `config.yaml` bağlam özelleştirmesi içindir; ana `specs/` başlangıçta boş olabilir.

`.agents/skills/openspec-*/SKILL.md`

Sohbet kontrolü

Asistanı aynı proje kökünde yeniden aç veya skill listesini yeniden yükle. Aşağıdaki keşif çağrısını **sohbete** gönder; terminale değil.

ASİSTAN SOHBETİ · seçilen proje/root

```
$openspec-explore
Bu projenin root'unu, okuduğun OpenSpec skill'ini ve
mevcut change listesini göster.
Kod veya plan belgelerini değiştirme.
```

Başarı nasıl görünür?

Asistan doğru root'u ve aktif OpenSpec change'lerini (henüz yoksa yok) söyler. İlgili skill'i kullanır; `openspec: command not found` veya genel yöntem açıklamasıyla yetinmez.

▼ User altında skill de olsun istiyorum

İlk kullanım için proje skill'i yeterli. CLI global; ürün verisi proje özelinde kalır. Codex'te kişisel skill keşif yolu `~/.agents/skills`. OpenSpec'i bir kişisel toolkit klasöründe init edip üretilen skill'leri buraya symlink etmek manuel bir seçenektir; ayrı bir `init --global` komutu varsayma.

Skill'ler yöntemi taşır, ürün A'nın domain bağlamını ürün B'ye taşımaz. Proje skill'i ile aynı adlı kişisel kopyayı bakım yapmadan çoğaltma. Tam kişisel kurulum reçetesi bu dosyanın atlasında:

▼ Başka bir asistan / eski sürüm?

`openspec init --help` ile kurulu CLI'ın desteklediği tool ID'lerini gör. Üretilen dosyalar ve init çıktısı esas. Üstteki seçim Codex, Claude Code ve Cursor için uyarlanır. Claude Code ile sıradan Claude web sohbeti aynı ortam değildir.

Varsayılan **core** yolunu kullan: explore / propose / apply / update / sync / archive. Expanded yol (new / continue / ff / verify / onboard vb.) için terminalde `openspec config profile`, ardından bu root'ta `openspec update`. Bu, asistan yönergelerini yeniler. İlk işi başlatmak için expanded gerekmez.

Geçiş kanıtı
Doğru root'ta üretilen skill görülüyor ve keşif çağrısı başarıyla çalışıyor.

TEK BAŞINA BAŞLA · 6/15 · mac · codex

En güçlü bağlamı kaynaklardan çıkar.

Ürün niyeti + ilgili kanıt + sürüm + açık karar. Bütün repo'yu sohbete dökmek bu dördünü sağlamaz.

PM'in vereceği

Kullanıcı/segment, sorun, beklenen sonuç, kapsam dışı, bilinen iş kuralları ve sahipleri. Ölçüm varsa olayın tanımı ve başlangıç verisi; yoksa "ölçülmedi". Domain = ürünün problem/iş kuralı alanı.

AI'ın çıkaracağı

Tek akışın ekran → istek → API → backend handler → domain kuralı → veri/olay → test rotası. Dosya ve sembolleri varsaymadan keşfeder. Kaynağın sürümünü belirtir.

ASİSTAN SOHBETİ · <…> alanlarını gerçek ürününle doldur

```
$openspec-explore
Ürün/domain: <...>; kullanıcı/segment: <...>.
Sorun ve beklenen sonuç: <...>; kapsam dışı: <...>.
Planlama root: <mutlak yol>.
Mobil: <checkout mutlak yolu>; branch/commit: <...>.
Backend: <checkout mutlak yolu>; branch/commit: <...>.
API, ürün kararı, tasarım, analitik ve release kaynakları: <dosya/bölüm/sürüm>.

Önce erişebildiğin kaynakları ve okunamayanları raporla.
Varsa docs/context/README.md ve sources.md oku; dizin ve stack'i keşfet.
Tek akışı ekran → istemci isteği → API → handler → domain → veri/olay → test
boyunca izle. Her bulguya repo/commit/dosya/sembol veya belge bölümü ekle.
Çalışma kopyasındaki commit dışı değişiklikleri ayrıca belirt.
Gözlem, onaylı kural, varsayım, çelişki ve bilinmeyeni ayrı yaz.
Kodun bulunması canlı deploy/flag kanıtı değildir. Bilinmiyorsa belirt.
Çıktı: kaynak sicili, sözlük, akış, kurallar, açık kararlar/sahipler,
etkilenen bileşenler ve kritik hata yolları. Kod değiştirme.
```

Bilgi türü	Örnek	Nasıl kullanılır?
Gözlem	Mobil timeout'ta hata gösteriyor.	Dosya/sembol + commit. Doğru ürün davranışı olduğuna tek başına karar vermez.
Onaylı kural	Doğrulanmayan sonucu başarısız göstermeyiz.	Karar kaydı + sahibi + tarih.
Varsayım	Talep 24 saatte uygulanır.	Kaynak yoksa onaylı kural gibi spec'e girmez.
Bilinmeyen	Canlı ortamda flag açık mı?	İlgili release/operasyon sahibine soru.
Çelişki	Doküman "anlık", kod "beklemede".	İki kaynağı da göster; beklenen davranışı sahibi seçer.

▼ Kod repolarına erişemiyorsam?

Ekran kaydı, API/OpenAPI çıktısı, akış belgesi, iş kuralı, anonimleştirilmiş destek örneği ve mevcut test/kabul kayıtlarıyla başlayabilirsin. Her belgenin tarih/sürümünü yaz. AI'a **"Bu paket implementasyonu kanıtlamaz; koddan doğrulanacakları ayrı listele"** de. Önce kullanıcı davranışının planını yaz; mühendislik teknik uygulanabilirlik ve kod eşlemesini tamamlasın.

> **Genel cevap geldiyse**
>
> "Bu iddiayı hangi dosyada, hangi sembolde gördün? Okuduğun ilgili bölümü ve sürümü göster. Erişmediğin kaynağı bulgu sayma." diye geri çevir. Dosya yolunu görmek dosyayı okumak değildir.

Geçiş kanıtı
En az bir akış iki taraf boyunca izleniyor; her iddia kaynaklı; okunamayanlar ve karar sahipleri ayrılmış.

10 Ekim 2026 · Keşif ↗ · Mevcut sistem ↗

TEK BAŞINA BAŞLA · 7/15 · mac · codex

Bağlamı kalıcı bir pakete dönüştür.

Konuşmada kalan bilgi bir sonraki PM'in veya yeni sohbetin başlangıcı olamaz.

Bu dizin yapısı **eğitim önerisi**; resmi zorunlu OpenSpec yapısı değildir. AI'ın ilk taslağını PM/domain sahibi/mühendislikle kontrol et, sonra planlama repo'sunda sürümle.

DOSYA DÜZENİ · planlama root'unda

```
docs/context/README.md   → okuma rotası, domain, sahip
docs/context/sources.md  → repo/belge, sürüm, durum
docs/context/domain.md   → terimler, onaylı kurallar
docs/context/flow.md     → akış, hata yolları, açık kararlar
openspec/config.yaml     → kısa bağlam + belge kuralları
AGENTS.md                → asistanın okuma protokolü
```

Küçük giriş, derin kanıt

`config.yaml` bütün repo'nun dump'ı değil. Ürünü ve okuma rotasını kısa anlatır; ilgili dosyaların gerçekten okunmasını ister. `context` bütün artifact yönergelerine eklenir. `rules` yalnız ilgili belge türüne yön verir.

Generated skill'lerin içine ürün bilgisi yazma; tool güncellemesi onları yenileyebilir. Domain bilgisi kendi sürümlenen dosyalarında yaşasın.

Bakım anlaşması

Her ortak kuralın sahibi olsun. Aynı kural her PM'e ayrı kopya verilmesin. Sözleşme, flag, ürün kararı veya ilgili kod değiştiğinde kaynak kaydını güncelle. Yeni işte bütün paketi körü körüne okumak yerine ilgili akışı seç.

ASİSTAN SOHBETİ · review edilmiş bulgulardan dosya üret

```
İncelenen bulguları docs/context/ dosyalarına kaynaklarıyla kaydet.
README: ürün, sahipler ve iş türüne göre okuma rotası.
sources: repo/branch/commit, belge sürümü, canlı ortam durumu.
domain: sözlük ve onaylı kurallar. flow: akış, hatalar, kararlar/sahipler.
```

```
openspec/config.yaml varsa mevcut schema/context/rules ile birleştir.
Yoksa spec-driven schema ile kısa context ve artifact rules öner.
AGENTS.md varsa mevcut talimatları koru; bağlam okuma protokolünü ekle.
Path'in içeriğinin otomatik yüklendiğini varsayma; ilgili dosyaları oku.
Şablon yer tutucularını gerçek bulguyla değiştir. Kod değiştirme.
```

Tüm şablonların içeriği atlas içinde okunabilir ve kopyalanabilir. ZIP de HTML'e gömülü; yan dosya gerekmez.

Geçiş kanıtı
Yeni sohbette README'den başlayınca aynı akış ve aynı karar kaynakları yeniden bulunabiliyor.

10 Ekim 2026 · Context, rules ve schema ↗

TEK BAŞINA BAŞLA · 8/15 · mac · codex

İlk işi küçült; ürün kararlarını koy.

“Paket değişikliğini düzelt” yerine tek bir gözlemlenebilir sonucu seç.

NİYET	SINIR	KABUL
Ne iyileşecek?	**Neler dışarıda?**	**Ne gözleyeceğiz?**
Yanıt kaybında kullanıcı yanlışlıkla başarısızlık görmesin.	Fiyat/uygunluk politikası, kampanya, geçmiş talep ekranı.	Aynı niyet ikinci talep yaratmıyor; doğrulanmış durum gösteriliyor.

Örnek ürün kararları: doğrulanmayan sonuç belirsiz gösterilir; kabul edilme ile uygulanma ayrılır; aynı kullanıcı niyeti tekrar gönderilince ikinci talep yaratılmaz. Bunlar bu eğitimin kurallarıdır, senin ürününe otomatik taşınmaz.

Karar henüz yoksa: soruyu, seçenekleri, etkisini ve karar sahibini yaz. İşin davranışını değiştiren karar açıkken AI’ın seçimiyle “hazır” demeyin. Teknik yaklaşımı mühendislik, kritik senaryoları QA ile inceleyin.

ASİSTAN SOHBETİ · planlama root’unda

```
$openspec-propose recover-package-change-result
Kaynak: incelenmiş docs/context/ paketi ve onaylı ürün kararları.
Amaç: yanıt alınamadığında belirsizliği doğru göstermek ve aynı niyetin
ikinci bir talep oluşturmasını önlemek.
Kapsam: sonuç gösterimi, aynı niyet, durum sorgusu ve ekrana dönüş.
Dışarıda: fiyat/fatura, uygunluk politikası, kampanya ve geçmiş ekranı.
Bu eğitimdeki ürün kurallarını gerçek ürüne otomatik taşıma.
Önce kaynakları ve sonucu belirleyen açık kararları göster.
Kritik karar açıksa sahibine soru; kararlaştırılmışsa dört belgeyi üret.
Senaryolar: kabul/yanıt kaybı, kesin ret, uygulanma, eşzamanlı tekrar,
kapanma/dönüş, yeni niyet, eski istemci ve yetki.
```

```
Proposal/spec/design/tasks tutarlılığını denetle; review için dur.
Uygulama koduna geçme.
```

Beklenen dosyalar

`openspec/changes/recover-package-change-result/` altında proposal.md, specs/package-change/spec.md, design.md ve tasks.md. Bu dört belge varsayılan spec-driven schema içindir; özel şema farklı olabilir.

Proposal neden/ne/kapsam, spec beklenen davranış, design teknik yaklaşım, tasks uygulama ve doğrulama işleri. Dört belgenin aynı değişikliği anlattığını kontrol et.

Geçiş kanıtı
Bir değişiklik kimliği, net kapsam, karar sahipleri ve senaryoları içeren dört tutarlı belge var.

TEK BAŞINA BAŞLA · 9/15 · mac · codex

Aynı işin bütün dosyalarını gör.

Kısaltılmış slayt metni yerine başlangıç spec'i ve dört belge yan yana.

Tek kimlik: `recover-package-change-result`. **Tek capability:** `package-change`. Önce başlangıç spec'i, sonra ona göre MODIFIED/ADDED delta. Böylece "hangi mevcut gereksinimi değiştiriyoruz?" sorusu cevapsız kalmaz.

> **Bu örneğin durumu: PLAN TASLAĞI**
>
> Örnek ürün kuralları net; mühendislik soruları açık. Format örneği olarak kullanılabilir. Gerçek bir ürün için uygulamaya hazır olduğuna veya test edildiğine ilişkin iddia yok.

▼ Kod repo'su olmadan ilk prova: bu örneği çalıştır

1. **Ayrı eğitim root'u aç.** Aşağıdaki kurulum yalnız prova içindir; gerçek ürün dosyalarının üzerine örnek spec kopyalama.

TERMINAL · yeni ve ayrı eğitim klasörü

```
mkdir -p "$HOME/work/openspec-practice"
cd "$HOME/work/openspec-practice"
openspec init --tools codex
```

2. ZIP'i aç. İçindeki `pm-starter-kit/training-example/openspec/` klasöründe `specs/package-change` ve `changes/recover-package-change-result` vardır.
3. Dosya yöneticisinde `package-change` klasörünü prova root'undaki `openspec/specs/` içine; `recover-package-change-result` klasörünü `openspec/changes/` içine kopyala. Init ile üretilen skill'leri koru. Örneklerin tam içerikleri aşağıda.
4. Aynı prova root'unda aşağıdaki kontrolleri çalıştır. Beklenen: aktif listede tek örnek change ve validate sonucunda ana spec + change için hata yok. Bu örnek gerçek OpenSpec CLI ile strict kontrolden geçirilmiştir.

TERMINAL · ayrı prova root'unda

```
openspec list
openspec show recover-package-change-result
openspec validate --all --strict
```

5. Asistanı prova root'unda aç; plan review mesajını gönder. Açık design sorularını buldur, ürün/teknik karar ayrımını yap. Bu prova belge üretimi/incelemesi içindir; gerçek ürün kodu uygulanmış veya test edilmiş sayılmaz.
6. Ürünün için yeni root'ta aynı yöntemi gerçek kaynaklarla tekrarla; eğitim örneğini gerçek ürün kuralı gibi kullanma.

▼ openspec/specs/package-change/spec.md

DOSYA · openspec/specs/package-change/spec.md

```
# Package Change

## Purpose
Paket değişikliği sonucunu kullanıcıya doğru durumla göstermek.
Bu başlangıç davranışı eğitim örneğidir, gerçek repo bulgusu değildir.

## Requirements

### Requirement: Gönderim sonucu
Sistem başarılı kabul yanıtında talebin alındığını göstermelidir (MUST).
Yanıt alınamazsa sistem genel hata göstermelidir (MUST).

#### Scenario: Kabul yanıtı alınır
- GIVEN kullanıcı uygun bir paket değişikliği gönderdi
- WHEN backend talebi kabul etti ve yanıt cihaza ulaştı
- THEN kullanıcı "Talebiniz alındı" görür

#### Scenario: Yanıt alınamaz
- GIVEN kullanıcı paket değişikliği gönderdi
```

- WHEN cihaz yanıt alamaz
- THEN kullanıcı genel hata görür

▼ **openspec/changes/recover-package-change-result/proposal.md**

DOSYA · openspec/changes/recover-package-change-result/proposal.md

```
## Why
Eğitim vakasında backend talebi kabul etse de yanıt kaybolabiliyor.
Genel hata kullanıcıyı aynı niyeti yeniden göndermeye yöneltiyor.
Beklenen sonuç: belirsizliği doğru gösterip ikinci talebi önlemek.

## What Changes
- Doğrulanamayan sonucu belirsiz göster; kesin ret olarak etiketleme.
- Kabul edilen ve uygulanması doğrulanmış talebi ayrı göster.
- Aynı niyetin tekrarını aynı talebe bağla.
- İlgili ekrana dönüşte doğrulanmamış talebi toparla.
- Kesin ret gerekçesini ve uygun devam yolunu göster.
Kapsam dışı: fiyat/fatura, uygunluk politikası, kampanya,
geçmiş talep ekranı ve yeni bildirim kanalı.

## Capabilities
### New Capabilities
- Yok.
### Modified Capabilities
- package-change: sonuç gösterimi, aynı niyet ve toparlanma davranışı.

## Impact
Mobil gönderim ve dönüş akışı; backend talep eşleme ve durum sorgusu;
API sözleşmesi; eski mobil sürüm uyumu; ilgili testler ve ölçüm.
```

```
Ürün kararları: bu eğitim için kararlaştırılmış örnek kurallar.
Gerçek işte kaynak/sürüm, karar sahipleri ve hedef ölçüm eklenir.
```

▼ **openspec/changes/recover-package-change-result/specs/package-change/spec.md**

DOSYA · openspec/changes/recover-package-change-result/specs/package-change/spec.md

```
## MODIFIED Requirements

### Requirement: Gönderim sonucu
Sistem yalnız doğrulanmış talep durumunu sonuç olarak göstermelidir (MUST).
Kabul edilmiş talebi "Talebiniz alındı", uygulanması doğrulanmış talebi
"Paketiniz uygulandı", doğrulanamayan sonucu "Sonuç kontrol ediliyor"
olarak göstermelidir (MUST). Kesin rette gerekçeyi ve uygun devam yolunu
sunmalıdır (MUST). Yanıt kaybı tek başına kesin ret değildir.

#### Scenario: Kabul yanıtı alınır
- GIVEN kullanıcı uygun paket değişikliği gönderdi
- WHEN backend talebi kabul etti ve yanıt cihaza ulaştı
- THEN "Talebiniz alındı" gösterilir
- AND paket uygulanmış gibi gösterilmez

#### Scenario: Yanıt alınamaz
- GIVEN talep gönderildi
- WHEN cihaz yanıt alamaz ve durum sorgusu da sonuç vermez
- THEN "Sonuç kontrol ediliyor" gösterilir
- AND kesin başarısızlık veya uygulanma gösterilmez

#### Scenario: Uygulanma doğrulanır
- GIVEN gönderilmiş bir talep var
- WHEN yetkili durum kaynağı uygulanmayı doğrular
```

- THEN "Paketiniz uygulandı" gösterilir

Scenario: Kesin ret doğrulanır
- GIVEN talep uygunluk nedeniyle kesin reddedildi
- WHEN yetkili durum kaynağı reti doğrular
- THEN uygunluk gerekçesi gösterilir
- AND kullanıcı paket seçimine geri dönebilir
- AND otomatik yeniden gönderim yapılmaz

ADDED Requirements

Requirement: Aynı niyetin tekrar gönderimi
Sistem aynı kullanıcı niyetinin tekrarında ikinci paket değişikliği oluşturmamalıdır (MUST). Kullanıcının farklı yeni niyeti bağımsız bir talep olarak ele alınmalıdır (MUST).

Scenario: Aynı niyet tekrar gönderilir
- GIVEN ilk gönderimin sonucu bilinmiyor
- WHEN aynı kullanıcı aynı niyeti tekrar gönderir
- THEN aynı talebin güncel durumu döndürülür
- AND ikinci talep oluşturulmaz

Scenario: Eşzamanlı tekrar
- GIVEN aynı niyetle eşzamanlı iki gönderim yapılır
- WHEN backend bu gönderimleri işler
- THEN en fazla bir talep kaydı oluşur
- AND iki yanıt aynı talebe bağlanır

Scenario: Yeni niyet
- GIVEN önceki talebin sonucu doğrulandı
- WHEN kullanıcı farklı bir paket seçip yeni değişikliği onaylar
- THEN bu niyet bağımsız talep olarak işlenir

```
### Requirement: Ekrana dönüşte toparlanma
Sistem ilgili ekrana dönen kullanıcı için önceki doğrulanmamış talebin
durumunu sorgulamalıdır (MUST). Durum yine doğrulanamazsa belirsiz
gösterimi korumalıdır (MUST).

#### Scenario: Uygulama yeniden açılır
- GIVEN talep gönderildikten sonra yanıt alınmadan uygulama kapandı
- WHEN aynı kullanıcı uygulamayı açıp paket değişikliği ekranına döner
- THEN önceki talebin durumu sorgulanır
- AND doğrulanırsa o sonuç gösterilir

#### Scenario: Dönüşte durum kaynağına erişilemez
- GIVEN önceki talebin sonucu doğrulanmadı
- WHEN kullanıcı döner fakat durum sorgusu sonuç vermez
- THEN "Sonuç kontrol ediliyor" korunur
- AND bu niyet için ikinci talep oluşturulmaz
```

▼ **openspec/changes/recover-package-change-result/design.md**

DOSYA · openspec/changes/recover-package-change-result/design.md

```
## Context
Temsili vakada istemci yanıt alamasa da sunucu kabul etmiş olabilir.
Bu eğitim taslağı bir çözüm yönü önerir; gerçek repo incelenmedi.
Ürün kabulü: delta spec. Uygulamaya geçiş için aşağıdaki açık teknik
kararlar mühendislikle tamamlanmalı.

## Goals / Non-Goals
Goals: aynı niyeti korumak, doğru durum göstermek, ekrana dönüşü toparlamak.
Non-Goals: fiyat/uygunluk politikasını değiştirmek, yeni geçmiş ekranı.
```

```
## Decisions (proposed; engineering review required)
- operationKey gönderimden önce istemcide oluşur, aynı niyette korunur.
  Yalnız yanıt içinde dönen requestId yeterli değil: yanıt kaybolabilir.
- Backend (authenticated user, operationKey) eşlemesini atomik tutar.
  Aynı key ve aynı payload aynı talebe bağlanır; farklı payload çelişki
  olarak ele alınır. Kesin davranış API sözleşmesinde netleşir.
- Yetkili durum sorgusu kabul / uygulanma / ret durumlarını ayırır.
- İstemci talep bağını kapanma/dönüş boyunca korur; hesap değişiminde
  başka kullanıcının talebini göstermez.

## Risks / Trade-offs
Eşzamanlı çift gönderim; key saklama süresi; eski istemci; geç gelen
sonuç; yetki sınırı; servis kesintisi; ölçümde aynı niyetin iki sayılması.

## Migration / Rollout
Backend uyumunu önce hazırla; eski mobil istemcileri ayrı doğrula.
Mobil sürüm/flag ve geri dönüş koşulunu ekipçe kararlaştır.
Sync/archive release yapmaz.

## Open Questions (block apply)
- Atomik eşlemenin mevcut altyapıdaki yöntemi ve testi nedir? Backend sahibi.
- Key saklama süresi / bitişte güvenli devam yolu nedir? PM + backend.
- Eski istemci ve payload uyumsuzluğu nasıl davranır? API + mobil sahipleri.
- Ret sözleşmesi, terminal durumlar ve yetki sınırı nedir? Domain + API.
- Hangi test ortamı, rollout/rollback ve gözlem koşulları? QA + release.
```

▼ **openspec/changes/recover-package-change-result/tasks.md**

DOSYA · openspec/changes/recover-package-change-result/tasks.md

```
## 1. Karar ve sözleşme
- [ ] 1.1 Design açık sorularını sahipleriyle çöz; spec ile hizala.
- [ ] 1.2 API durum/payload/ret/yetki sözleşmesini sürümle.
- [ ] 1.3 Eski istemci uyumu ve release/rollback sırasını kararlaştır.

## 2. Backend (yalnız kararlar tamamlandıktan sonra)
- [ ] 2.1 Aynı niyet eşlemesini atomik uygula; yeni niyeti ayrı tut.
- [ ] 2.2 Yetkili durum sorgusunu ve payload çelişkisini uygula.
- [ ] 2.3 Tekrar/eşzamanlılık/yetki testlerini çalıştır, kanıt kaydet.

## 3. Mobil
- [ ] 3.1 Niyet anahtarını ilk gönderimden önce üret ve dönüşte koru.
- [ ] 3.2 Belirsiz / alındı / uygulandı / ret gösterimlerini ayır.
- [ ] 3.3 Dönüş, hesap değişimi ve erişilemeyen durum senaryolarını doğrula.

## 4. Kabul ve kapanış
- [ ] 4.1 Tüm delta senaryolarını ortam/sürüm/test bağlantısıyla kaydet.
- [ ] 4.2 Başarısız ve çalıştırılmamış senaryoları görünür tut; PM/QA review.
- [ ] 4.3 Deltayı sync et; ana spec'i incele; tamamlanan işi archive et.
- [ ] 4.4 Gerçek release ve gözlem kaydını ayrı yürüt.
```

Delta nasıl seçilir?

Başlangıçtaki `Gönderim sonucu` var: MODIFIED altında **tam yeni hâli** ve senaryoları yazılır. Aynı niyet / dönüş gereksinimleri yok: ADDED. Mevcut gereksinimi değiştirirken senaryo başlıklarını da koru; bu örnekte CLI kaybolan senaryoyu hata sayar. Değişmeyen diğer gereksinimleri delta içine kopyalama. Silme gerekiyorsa REMOVED ve gerekçe; capability'yi tamamen kaldırma özel işlemidir, atlasın kapanış bölümünde bulunur.

Geçiş kanıtı
Başlangıç → delta eşleşmesini gösterebiliyor ve açık tasarım kararları yüzünden apply için neden durulduğunu açıklayabiliyorsun.

10 Ekim 2026 · Requirement ve delta biçimi ↗

TEK BAŞINA BAŞLA · 10/15 · mac · codex

Planı sorgula; değişince hizala.

AI'ın düzgün yazması, ürün kararlarının doğru olduğu anlamına gelmez.

PM + domain review

1. Bu problem kimde, hangi kanıtla var?
2. Başarı, kapsam dışı ve açık kararlar görünür mü?
3. Her requirement gözlemlenebilir mi?
4. Belirsiz sonuç, kesin ret ve uygulanma ayrılıyor mu?
5. Kural ve süreler kaynaklı mı?

Mühendislik + QA review

1. Spec ile design aynı davranışı mı anlatıyor?
2. Eski mobil, eşzamanlılık, yetki ve dönüş kapsanıyor mu?
3. Her kritik senaryonun task/test karşılığı var mı?
4. Bağımlılıklar ve release sırası biliniyor mu?
5. Uygulama öncesi çözülmesi gereken soru kaldı mı?

ASİSTAN SOHBETİ · bir review talebi, apply değil

```
recover-package-change-result planını eleştirel incele.
Her bulguyu belge/satır, kaynak veya eksik kararla ilişkilendir.
Uydurulmuş kural, kapsam sızıntısı, eksik hata yolu, çelişki,
eski istemci/yetki/eşzamanlılık ve test boşluklarını önem sırasıyla yaz.
PM kararı, mühendislik kararı ve QA kanıtını ayır.
Kritik açıkları çözmeden uygulamaya hazır deme. Kod değiştirme.
```

TERMINAL · planlama root'unda

```
openspec validate recover-package-change-result --strict
```

Başarı: CLI yapısal hata bildirmiyor. Bu, ürün kurallarının doğruluğunu ve kodun çalışmasını kanıtlamaz. Hata mesajındaki dosya, başlık veya eksik scenario'yu düzeltip yeniden çalıştır.

İki update, iki farklı iş

Terminalde `openspec update` asistan yönergelerini yeniler. Sohbetteki plan update çağrısı mevcut change belgelerini değiştirir. Kod zaten uygulandıysa planı güncellemek kodu kendiliğinden uyarlamaz.

ASİSTAN SOHBETİ · plan değişikliği

```
$openspec-update-change recover-package-change-result
Yeni/onaylanan karar: <karar + kaynak + sahibi>.
Proposal, delta specs, design ve tasks'i bu kararla hizala.
Etkilenen senaryoları ve varsa önceden uygulanmış kodun uyarlama ihtiyacını
raporla. Bu çağrıda kod değiştirme.
```

Elle düzenleme de olur. Markdown dosyasını değiştir, sonra asistana bu yeni hâli okutup proposal/spec/design/tasks çelişkilerini düzeltmesini iste. Yeni bağımsız sonuç için ayrı change aç; aynı değişiklikte öğrenilen kararları update et.

Geçiş kanıtı
Format geçerli; kritik ürün/teknik kararlar çözülmüş; ekip planın aynı niyeti taşıdığını doğruluyor.

TEK BAŞINA BAŞLA · 11/15 · mac · codex

Uygula; gerçek sonucu kanıtla.

Task kutusu, test sonucu ve ürün kabulü ayrı kayıtlardır.

Planlama root'unda yalnız belgeleri değiştiren PM, kodu ilgili mühendislik sahibine devreder. Monorepo'da aynı root'ta devam edilebilir. Ayrı repolarda mobil/backend oturumları kendi implementation root'unda çalışır; ortak planın konumu ve sürümü her oturuma verilir.

ASİSTAN SOHBETİ · component root'unda mühendislik devri

```
Bu component root: <mobil veya backend mutlak yolu>.
Ortak plan: <planning root>/openspec/changes/recover-package-change-result/.
Plan sürümü: <commit>; bu component kaynak sürümü: <commit>.
Ortak proposal/spec/design/tasks dosyalarını gerçekten oku.
Bu component'e ait işleri ve bağımlılıkları ayır. Başka repo'ya yazma.
Yerel OpenSpec change/reference gerekiyorsa ekip düzenine göre oluştur;
ortak ürün kurallarını çelişen bağımsız kopyaya dönüştürme.
Uygulama öncesi açık karar/erişim sorunu varsa raporla.
Sonuçta task → requirement → test kanıtı eşlemesini ver.
```

ASİSTAN SOHBETİ · uygulama sahibi, doğru implementation root'unda

```
$openspec-apply-change recover-package-change-result
Onaylanmış plan ve bu component'in görevleriyle ilerle.
Önce root'u, kaynak/plan sürümlerini ve açık karar olmadığını doğrula.
Çalıştırdığın testleri ortam/komut/sonuçla kaydet.
Başarısız veya çalıştırılmayanı geçti olarak yazma.
```

Apply, planın görevlerini uygulamaya geçirir. Kod ve test komutları ürünün stack'ine göre seçilir; OpenSpec ortak bir `npm test` zorunluluğu getirmez. Test yalnız okunmuşsa "çalıştırıldı" denmez.

Kritik senaryo	Beklenen	Kanıt kaydı
Backend kabul, yanıt kayıp	Belirsiz gösterim; ikinci talep yok.	Test adı, ortam, mobil/backend sürümü, gözlenen sonuç.
Eşzamanlı aynı niyet	En fazla bir talep.	Gerçek concurrency testi ve kayıt sayısı.
Kapanma → dönüş	Önceki niyetin durumu sorgulanır.	Mobil senaryo kaydı; yetkili yanıt.
Kesin ret / uygulanma	Doğru gerekçe / doğru sonuç.	İki ayrı kabul senaryosu.
Eski istemci / başka kullanıcı	Uyum ve yetki korunur.	Regression ve yetki testleri.

`DOSYA · kabul kaydı örneği; sonuçları gerçekten doldur`

```
Senaryo | Requirement | Ortam | Mobil/backend commit | Test/demo bağlantısı
Gözlenen sonuç | Durum: geçti / başarısız / çalıştırılmadı | Sahip | Tarih

Bir senaryonun geçmesi bütün senaryoların geçtiği anlamına gelmez.
Başarısız veya çalıştırılmayan kritik yolun kararı görünür kalır.
```

▼ Verify kullanmalı mıyım?

Expanded profilindeki verify, doküman ile uygulama uyumunu inceler. Faydalı ek review; test çalıştırmanın veya PM kabulünün yerine geçmez. Archive için OpenSpec'in zorunlu kilidi değildir. Açmak için terminalde `openspec config profile`, sonra `openspec update`. Üretilen verify skill/komutunu sohbetten kullan.

Geçiş kanıtı
Kritik kabul senaryolarının gerçek kayıtları var; başarısız/çalıştırılmayanlar gizlenmiyor; ürün ve teknik sahipler sonucu incelemiş.

10 Ekim 2026 · Apply ve verify ↗ · Kabul incelemesi ↗

TEK BAŞINA BAŞLA · 12/15 · mac · codex

Sync, archive, release: üç ayrı sonuç.

Kapanıştan sonra yeni PM hangi davranışın referans olduğunu bulabilsin.

SYNC	ARCHIVE	RELEASE
Ana spec'i günceller	**İşi geçmişe taşır**	**Ürünü yayımlar**
Change delta'sı specs/ referansına işlenir.	Karar ve uygulama planı arşivde korunur.	Backend deploy, mobil dağıtım, flag ve gözlem ayrı süreç.

ASİSTAN SOHBETİ · planlama root'unda

```
$openspec-sync-specs recover-package-change-result
İncelenen deltayı ana spec'e işle; değişen requirement'ları göster.
Değişmeyen kuralların korunduğunu kontrol et.
```

Sync sonucu `openspec/specs/package-change/spec.md` dosyasını açıp tam yeni gereksinimleri incele. Gerekirse ayrı sync yapmadan archive akışı içinde sync teklifi gelir; yine sonucu kontrol et.

ASİSTAN SOHBETİ · kabul kanıtı incelendikten sonra

```
$openspec-archive-change recover-package-change-result
Önce kalan görevleri, senaryoların kanıtını ve sync durumunu özetle.
Eksik kritik kabul varsa gizleme. Tamamlanan işi arşivle.
Ana spec ve arşiv konumunu göster. Release/deploy yapıldığı iddiası kurma.
```

TERMINAL · planlama root'unda

```
openspec list
openspec validate --all --strict
git status --short
git diff
```

Beklenen: tamamlanan change aktif listeden çıkmış; `openspec/changes/archive/` altında tarihli kaydı var; ana spec beklenen yeni hâlde; Git diff istenen dosyaları gösteriyor. `git diff` yalnız izlenen dosyaları gösterir; yeni/untracked dosyaları ayrıca açıp incele.

Ekiple nasıl paylaşılır?

Planlama repo'sunda bağlam + change + spec dosyalarını ekipçe kullanılan branch/PR akışıyla paylaş. Component PR'ları ortak change kimliğine ve plan sürümüne bağlansın. Biri davranışı değiştirince kaynağı ve kabul kaydı da güncellensin. Git'e yazmak veya archive etmek canlıya çıkmak değildir.

ASİSTAN SOHBETİ · yarın yeni oturumda

```
Planlama root: <mutlak yol>. Önce docs/context/README.md ve sources.md oku.
Sonra openspec list ile aktif change'leri göster.
Devam edeceğimiz change: <kimlik>; varsa proposal/spec/design/tasks oku.
Son kaynak sürümleriyle değişen/eskimiş bağlamı raporla.
Tamamlanan, kalan ve açık kararları özetle; sonraki somut adımı öner.
Kaynak okumadan eski sohbetten ürün kuralı varsayma.
```

Yeni işe geçerken değişen bağlamı refresh et. Her küçük metin düzeltmesine dört belge açmak gerekmeyebilir; ekip için kalıcı davranış kararı veya riskli değişiklik varsa bu düzenin maliyeti karşılığını verir.

Geçiş kanıtı
Ana spec, arşiv ve paylaşım kayıtları kontrol edilmiş; release durumu ayrı belirtilmiş; yeni sohbet işi yeniden bulabiliyor.

Takıldığında: belirti → teşhis → çözüm.

Hata mesajını değiştirmeden, hangi adımda çıktığını belirterek ilerle.

▼ node/npm bulunamadı veya sürüm eski

Terminali yeniden aç; sürümleri tekrar kontrol et. Aktif Node yöneticin varsa desteklenen sürümü seç. Yoksa ortam adımındaki kurulum rotasını izle. GUI uygulamasının terminali eski ortamı taşıyorsa uygulamayı yeniden aç.

▼ openspec bulunamadı

Kurulum başarılı mı? Home alternatifi kullandıysan PATH doğru mu? Yeni terminalde de çalışıyor mu? macOS/Linux: `command -v openspec`; Windows: `Get-Command openspec`. Birden fazla Node/CLI varsa beklenen executable'ı kullandığını kontrol et. Home prefix için açık executable yoluyla sürümü çalıştır; çalışırsa sorun PATH'tir.

▼ EACCES / permission denied / kurulum yetkisi

İzin hatası sistem global hedefinden geliyorsa kullanıcı home alternatifine geç. Mevcut Node yöneticisi varsa onun kurulumunu kullan. Kurum kısıtıysa BT'den izinli yöntem iste. Dosyalar doğru kullanıcıya mı ait kontrol et; hata mesajını ve kurulum yolunu paylaş.

▼ Ağ / proxy / sertifika / oturum erişimi

Hangi host/servis erişilemediğini hata çıktısından ayır: npm indirmesi, repo clone veya AI hesabı. Kurum proxy/sertifika ve repo yetkisini ilgili sahibiyle çöz. Başarısız indirme tamamlanmış kurulum sayılmaz; erişim düzeldikten sonra sürüm ve okuma kontrolünü tekrar yap.

▼ Skill görünmüyor / sohbet komutu tanınmıyor

Doğru root ve doğru tool ID ile init yaptın mı? Üretilen dosya var mı? Asistanı aynı projede yeniden aç. CLI güncellendiğinde projede `openspec update` çalıştır. Codex'te dolar skill adı, Claude'da iki noktalı slash, Cursor'da tireli slash kullanılır. Verify gibi expanded skill default core'da yok.

▼ AI dosyayı okuyamıyor veya çok genel konuşuyor

Yerel checkout var mı? Asistanın o klasöre erişimi var mı? Repo URL'si/ChatGPT proje adı tek başına yerel erişim değildir. Mutlak yol ver, ilgili dosyanın bölümünü ve commit'i okut. Okuyamıyorsa eksport belgelerle çalışıp eksik kanıtı işaretle. Büyük repo'da tek akış ve küçük okuma rotası seç.

▼ Yanlış projede değişiklik üretti

Yeni kod üretimini durdur. Asistana CWD/root ve yazdığı dosyaları raporlat; istenmeyen dosyaları birlikte incele. İlgisiz çalışmayı silmeden doğru root'ta yeniden aç. Birden fazla store/global default varsa hedefi netleştir. Root'lar ve ürünler birbirine karışmasın.

▼ Validate hata verdi

CLI çıktısındaki dosyayı aç: ADDED/MODIFIED bölümü doğru mu? `### Requirement:`, `#### Scenario:`, MUST/SHALL ve en az bir senaryo var mı? MODIFIED başlığı ana spec'teki requirement adıyla eşleşiyor mu? Tam yeni requirement yazılmış mı? Mevcut senaryoların başlıkları korunmuş mu? Düzelttikten sonra aynı validate komutunu tekrar çalıştır.

▼ Belgede "tamamlandı" ama test yok

Durumu "çalıştırılmadı" olarak düzelt. Hangi requirement için hangi test/ortam/sürüm gerektiğini QA/mühendislikle belirle. Format geçmesi ve task işaretlemek kabul kanıtı değildir. Kritik eksik doğrulamanın sahibi ve sonraki adımı görünür olsun.

▼ Plan değişti; kod eski davranışta kaldı

Update planı değiştirir. Etkilenen design/tasks/spec ile uygulamayı yeniden hizala; ilgili root'ta apply ve regression doğrulaması yap. Sırf belgeler

güncel diye iş tamamlandı sayılmaz.

▼ Örnekteki komut sürümümde yok

`openspec --version` ve `openspec --help` çıktısını al. Core yolu tercih et. Beta store/workset/context komutlarını ilk işin ön koşulu yapma. Kurulu sürüm desteklemiyorsa ayrı planlama repo'su ve açık dosya referanslarıyla ilerle. Asistan yönergeleri ile CLI sürümü farklıysa projede update ile yenile.

ASİSTAN SOHBETİ · teşhis için doldur

```
OS ve terminal: <...>
Node/npm/OpenSpec sürümleri: <...>
Asistan ve proje root: <...>
Çalıştırdığım tam komut veya sohbet mesajı: <...>
Hata çıktısı: <kişisel/secret bilgileri çıkararak aynen>
Beklenen: <...>; Gözlenen: <...>
Yaptığım kontroller: <...>
Önce kök nedeni ayır, sonra en küçük düzeltmeyi öner.
Başarı kontrolünü tekrar çalıştır; ilgisiz dosyaları değiştirme.
```

Geçiş kanıtı
Sorunu doğru katmana bağlayabiliyor; çözümden sonra aynı başarı kontrolünü tekrarlayabiliyorsun.

Terim sözlüğü ve komut kartı.

Anlaşılmayan terim varsa cümleyi ezberleme; burada işiyle ilişkilendir.

Terimde veya açıklamada ara

OpenSpec

AI kod asistanıyla davranış gereksinimi ve değişiklik planını sürümlenen dosyalarda yönetme aracı.

Terminal / shell

Komut yazdığın arayüz / o komutu yorumlayan program. macOS'ta çoğu kullanıcı zsh, Windows'ta PowerShell kullanır.

CLI

Command-line interface: terminalden çalışan araç. openspec komutu bunun örneği.

Node.js / npm

Node çalıştırma ortamı; npm paket kurucusu. Asistan veya domain paketi değildir.

Global / user / project

Global CLI her klasörden çağrılabilir; user ayar/skill kullanıcıya aittir; project bağlam/spec o ürüne aittir.

Home / ~ / $HOME

Kullanıcı ana klasörü. ~ ve $HOME Unix shell kısaltmaları; PowerShell'de $HOME kullanıcı klasörünü verir.

Prefix

npm kurulumunun üst hedef dizini. Unix'te komutlar genelde prefix/bin; Windows'ta prefix altında.

PATH

Shell'in executable aradığı dizin listesi. Paket kurulmuş olsa da PATH eksikse komut bulunamayabilir.

Executable
İşletim sisteminin çalıştırdığı program veya komut dosyası. Hangi kopyanın çağrıldığı önemlidir.

Root / CWD / mutlak yol
Root projenin ana klasörü; CWD mevcut çalışma dizini. Mutlak yol konumu tam belirtir; göreli yol CWD'ye bağlıdır.

Repo / checkout / clone
Repo sürümlenen proje; checkout bilgisayarındaki çalışma kopyası; clone uzaktaki repo'dan bu kopyayı oluşturur.

Branch / commit / diff / PR
Branch çalışma hattı; commit kayıtlı sürüm; diff dosya farkı; PR ekip incelemesine sunulan değişiklik.

Domain
Ürünün problem alanı ve iş kuralları. Örnek: ödeme, abonelik, kimlik doğrulama.

Context / bağlam
AI'ın işi doğru yorumlaması için ürün niyeti, kaynaklar, kurallar, sürümler ve kararlar.

Skill / slash command
Asistanın belirli çalışma yöntemini çağırma yolu. Codex skill'i $openspec-...; diğer asistanlarda /opsx biçimleri.

AGENTS.md
Asistana proje çalışma talimatı veren dosya. Mevcut talimatlar korunarak okuma protokolü eklenir.

Schema / artifact / rules
Schema belge türleri ve bağımlılıklarını tanımlar; artifact bir çalışma belgesi; rules belgeye özel yönergeler.

Proposal
Neden bu değişiklik, ne değişecek, kapsam ve etki. Ürün hedefinin başlangıç belgesi.

Spec / requirement / scenario

Spec davranış sözleşmesi; requirement beklenen bir davranış; scenario belirli koşul ve gözlemlenebilir sonuç.

Capability

Spec'lerin düzenlendiği davranış/yetenek alanı; bu örnekte package-change. Klasör adı ve proposal eşleşmelidir.

Delta / ADDED / MODIFIED / REMOVED

Mevcut spec'e göre davranış farkı. Yeni gereksinim / tam yeni sürüm / gerekçeli kaldırma.

GIVEN / WHEN / THEN

Başlangıç koşulu / olay / beklenen sonuç. Belirsiz "düzgün çalışır" yerine gözlemlenebilir sonuç yaz.

MUST / SHALL / SHOULD / MAY

Zorunlu / zorunlu / öneri / isteğe bağlı. Ürün gereksiniminin bağlayıcılık dilini netleştirir.

Design / tasks

Design teknik çözüm ve riskler; tasks uygulama ve doğrulama işlerinin listesi. Figma tasarımıyla aynı anlamda değil.

Explore / propose / apply / update

Keşif / plan üretimi / uygulama / plan güncelleme. Terminaldeki openspec update asistan yönergelerini yeniler.

Validate / verify / kabul

Validate formatı kontrol eder; verify belge-kod uyumunu inceler; kabul gözlenen ürün davranışına dayanır.

Sync / archive

Sync deltayı ana spec'e işler; archive change'i geçmişe taşır. Ürünü yayımlamaz.

API / endpoint / handler

Bileşenler arası sözleşme / çağrılan adres+işlem / backend'de isteği karşılayan kod.

Timeout / retry / idempotency

Yanıt zamanında gelmedi / yeniden dene / aynı niyetin tekrarında ek iş etkisi oluşmaması. Timeout kesin ret değildir.

operationKey / requestId

Bu vakada gönderimden önce oluşturulan niyet anahtarı / sunucu talep kimliği. Yanıtta dönen ID, yanıt kaybında tek başına yeterli olmayabilir.

Atomiklik / concurrency

İşin bölünmez tamamlanması / aynı anda gerçekleşen işler. Tek talep sözü eşzamanlı tekrar testini gerektirir.

Deploy / release / flag / rollback

Backend dağıtımı / kullanıcıya sürüm sunma / davranışı açan ayar / önceki güvenli hâle dönüş.

Store / workset / beta

Store paylaşılan plan konumu; workset kişisel çalışma klasörleri grubu. Beta seçenekler ilk iş için zorunlu değil; kod erişimi otomatik oluşmaz.

Kaynak / sürüm / kanıt

İddiayı nerede gördüğün / hangi kayıtlı hâlini incelediğin / davranışı destekleyen gözlem veya gerçek test sonucu.

Günlük komut kartı

TERMINAL · planlama root'unda

```
openspec --version
openspec list
openspec show <change-kimliği>
openspec validate <change-kimliği> --strict
openspec update  # asistan yönergelerini yeniler
```

ASİSTAN SOHBETİ · seçilen tool biçimi

```
$openspec-explore
$openspec-propose <change-kimliği>
$openspec-apply-change <change-kimliği>
$openspec-update-change <change-kimliği>
$openspec-sync-specs <change-kimliği>
$openspec-archive-change <change-kimliği>
```

Terminal komutları shell'de, skill/slash çağrıları asistan sohbetinde. Başlangıç için core yeterli. İleri komutlar atlasın ilk bölümünde.

Geçiş kanıtı
CLI/skill/context farkını ve validate/sync/archive farkını kendi cümlenle anlatabiliyorsun.

10 Ekim 2026 · CLI ↗ · Komutlar ↗

TEK BAŞINA BAŞLA · 15/15 · mac · codex

Hissetme; yapabildiğini göster.

Sekiz soru + kendi domain'inde bir iş. Yanlış cevabın gerekçesi de burada.

Her soruya cevap ver; sonra kontrol et. Amaç kelime ezberi değil, yanlış karar vermeyi önlemek.

1. CLI home'a kuruldu. Ürün A ve B'nin bağlamı nereye gider?

A. Tek bir global context'e.
B. Her ürünün kendi planlama root'una.
C. Generated skill dosyasının içine.

Yanıt: B. Global kurulum yöntemi ortaklaştırır. Ürün bağlamı/spec farklı root'larda kalır.

2. Backend talebi kabul etti, mobil yanıt alamadı. Kesin olan ne?

A. Talep başarısızdır.
B. Paket uygulanmıştır.
C. Mobil sonuçtan emin değildir; sunucu durumu doğrulanmalı.

Yanıt: C. Yanıt kaybı istemci bilgisidir. Kabul ve uygulanma ayrı durumlar; güvenilir kaynaktan kontrol gerekir.

3. config.yaml'a bir dosya yolu yazdın. Bu neyi garantiler?

A. Dosyanın bütün içeriği okundu.
B. Sadece bir okuma referansı verildi; okuma kanıtı ayrıca gerekir.
C. Kaynak günceldir.

Yanıt: B. Path görünmesi, dosyanın okunması ve sürümünün doğru olması aynı şey değil.

4. Mevcut Gönderim sonucu requirement'ını değiştiriyorsun. Delta?

A. MODIFIED altında tam yeni requirement ve senaryoları.
B. ADDED altında yalnız değişen cümle.
C. Ana spec'i sil ve archive et.

Yanıt: A. MODIFIED aynı adlı mevcut gereksinimi tam yeni hâliyle değiştirir; kopuk tek cümle yeterli değildir. Mevcut senaryo adlarını koru; bu örnekte gerçek CLI doğrulaması ad değişimini eksik senaryo saydı.

5. Validate geçti. Ne kanıtlandı?

A. Tüm mobil/backend testleri geçti.
B. Ürün yayımlandı.
C. Yapısal/format kontrolü geçti.

Yanıt: C. Test ve ürün kabulü gerçek davranış kanıtı gerektirir. Format kontrolü bunları çalıştırmaz.

6. Sohbette update ile plan değişti, daha önce kod uygulanmıştı. Sonraki adım?

A. Kod kendiliğinden değişti.
B. Etkilenen kod/tasks'i apply ve test ile yeniden hizala.
C. Hemen archive et.

Yanıt: B. Plan düzenlemesi uygulanmış kodu kendiliğinden değiştirmez.

7. QA kritik dönüş senaryosunu çalıştırmadı. Durum?

A. Çalıştırılmadı; sahibi ve sonraki adımı görünür.
B. Tasks işaretliyse geçti.
C. AI olumlu yazdıysa geçti.

Yanıt: A. Kanıt yoksa passed yok. İncelenmiş test dosyası çalıştırılmış test değildir.

8. Archive başarılı. Canlı release durumu?

A. Bütün kullanıcılar yeni davranışta.
B. Backend deploy oldu; mobile gerek yok.
C. Ayrı deploy/app release/flag kanıtıyla belirlenir.

Yanıt: C. Archive dosya geçmişini yönetir. Canlı yayın ve sürüm uyumu ayrı süreçler.

Asıl bitirme görevi: kendi ürününde tek iş

Bir akış seç. Kaynak ve sürümleri kaydet. Bir gözlem, bir ürün kuralı, bir açık soru yaz. Kapsam dışını belirle. En az bir mutlu yol, bir hata yolu ve bir tekrar/dönüş senaryosuyla değişiklik planını hazırla. Mühendislik/QA ile kabul kanıtının nasıl üretileceğini bağla.

DOSYA · doldurulacak bitirme / devir kartı

```
Ürün + akış + kullanıcı:
Planlama root / change kimliği / plan commit:
Mobil / backend kaynak sürümleri:
Kaynaklı mevcut davranış:
Onaylı beklenen davranış + karar sahibi:
Kapsam dışı:
Açık karar + sahibi + sonraki adım:
```

```
Proposal/spec/design/tasks yolu:
Kritik senaryolar + test/demo sahibi:
Kabul kaydı: geçti / başarısız / çalıştırılmadı:
Sync / archive kontrolü:
Release durumu (ayrı):
```

> **Bitiş ölçütü**
>
> Soruların doğru gerekçelerini açıklıyor ve kendi işinin kartını gerçek kaynaklarla doldurabiliyorsan bağımsız başlangıç yapabilirsin. Quiz sonucu, gerçek kurulum veya uygulama başarısını otomatik ölçmez.

Bu kartı doldurmak için eğitmene ihtiyaç yok; ürünün bilinmeyen kararlarında ilgili sahibine ihtiyaç var. OpenSpec karar sahibinin yerine karar vermez.

Geçiş kanıtı
Bitirme kartı gerçek bir iş için dolu; eksikler sahibine bağlı; plan/kabul/release ayrımını koruyorsun.

10 Ekim 2026 ·